Botho from the Basics

A teaching companion to the Botho whitepaper

Contents

Botho from the Basics

A teaching companion to the Botho whitepaper. Everything below is taught from intuition. For the formal treatment of any mechanism — the exact equations, the security proofs, the normative parameter tables — this primer points you to the relevant section of the whitepaper rather than restating it. When the two documents seem to disagree, the whitepaper wins.

Who this is for

You are technically curious but not a cryptographer. Maybe you are a developer thinking about building on Botho, a prospective node operator, or a user who wants to actually understand what happens when you press "Send." We assume you are comfortable with software-level ideas — you know roughly what a hash is, what a public/private key pair is, and what a digital signature does, at the "black box" level. We do not assume you know elliptic curves, commitment schemes, lattice cryptography, or the Byzantine fault tolerance literature. Every piece of jargon is introduced before it is used.

Two kinds of material get different treatment here:

- Standard privacy building blocks — stealth addresses, ring signatures, confidential amounts. These come from the CryptoNote/Monero lineage and have excellent external tutorials (the

Monero project's *Zero to Monero* is the canonical deep dive). We teach the intuition you need and cite out the rest.
- The pieces that are novel to Botho — the hybrid post-quantum stealth address, the pairing of Stellar-style consensus with proof-of-work mining, and the anti-hoarding economics. No external tutorial exists for these, so they get the most ink.

The capstone follows a single payment end to end through every mechanism in the primer.

---

1. The problems, before any machinery

Every mechanism in Botho is an answer to one of three problems. Hold on to these; each later section will point back at them.

Problem 1: Public ledgers are surveillance machines

A blockchain needs everyone to agree on who owns what, and the obvious way to get agreement is to make every transaction public. Bitcoin does exactly this, and the result is a permanent, searchable record of every payment ever made. Once any address is linked to your identity — an exchange withdrawal, a payment to a friend, a donation — an observer can walk the transaction graph in both directions. Entire companies exist to do this at scale.

So a private currency has to hide three things at once:

1. Who received the money (otherwise your address is a lifelong tracking beacon),
2. who sent it (otherwise your spending history is public), and
3. how much moved (otherwise amounts alone often identify you).

Sections 3–5 of this primer cover the three tools that hide these three things. Crucially, the hiding cannot be optional: if privacy is a feature only the cautious turn on, using it becomes suspicious in itself, and the "anonymity set" — the crowd you hide in — stays small. Botho makes all three protections mandatory for every transfer.

Problem 2: The chain lives forever, and quantum computers are coming

Blockchain data is permanent and public. That combination creates a threat that most systems never face: harvest now, decrypt later. An adversary can record today's encrypted or key-derived data and simply wait. If a large quantum computer arrives in twenty years, it can retroactively break the elliptic-curve math protecting today's recorded data — and the blockchain, unlike a TLS session, will still be sitting there in full.

For classical-only privacy coins (Monero, and also MobileCoin, the system closest to Botho's design), this means a future quantum adversary could retroactively unmask recipients across the entire recorded history of the chain. Botho's answer — protect *permanent* data with post-quantum cryptography, and keep efficient classical cryptography where the secret's value is *ephemeral* — is Section 6 of this primer, and it is one of the places where Botho genuinely differs from everything before it.

Problem 3: "Number go up" economics concentrates wealth

Fixed-supply currencies reward early holders for doing nothing but holding. Beyond the fairness question, there is a security problem: when block rewards taper to zero, miners must be paid entirely from

fees, and it is an open question whether fees alone can fund adequate security. And there is a usability problem: deflationary money discourages spending — the very activity a currency exists for.

Botho's design philosophy (the name comes from the Sesotho/Setswana word for *humanity*; the whitepaper opens with the proverb "a person is a person through other people") asks a different question: how can money serve circulation rather than accumulation? Its answer is a coordinated set of economic mechanisms — perpetual but modest emission, fees that scale with the wealth of the coins being moved, a holding cost on concentrated idle wealth, and a per-block lottery tilted toward small holders. That is Section 9, the largest section of this primer.

> For the whitepaper's own framing of these three problems, see §1 (Introduction) and §2 (Related Work).

---

2. A ten-minute toolbox

You need exactly four ideas from cryptography. Each is stated as a usable black box; the whitepaper's §3 (Preliminaries) gives the formal versions.

Hashes. A hash function turns any input into a fixed-size fingerprint. It is one-way (you cannot recover the input) and collision-resistant (you cannot find two inputs with the same fingerprint). Botho, like most modern protocols, also uses hashes as a *randomness beacon*: the hash of unpredictable data is itself unpredictable, and everyone who sees the data computes the same value.

Scalars and points: math you can only do forward. Botho's classical cryptography lives on an elliptic curve (a group called Ristretto255 — the name doesn't matter here). You need only this picture: there are *points* (public things) and *scalars* (secret numbers). You can multiply a point by a scalar to get another point, and you can add points together. But you cannot divide: given the starting point G and the result $x \cdot G$, no known classical algorithm recovers x. This one-way multiplication is the entire basis of classical public-key cryptography: your private key is a scalar x, your public key is the point $x \cdot G$. A useful mental model is mixing paint: combining colors is easy, un-mixing is not.

The catch — central to Section 6 — is that this one-wayness holds only for *classical* computers. Shor's algorithm on a large quantum computer inverts it efficiently. Everything built on curve points is quantum-fragile.

Signatures. A digital signature proves "the holder of the private key behind this public key approved this exact message," without revealing the private key. You have used these; the only new twist Botho adds is a signature that proves the signer is *one of a group* without saying which one (Section 4).

Key encapsulation (KEM). A KEM is the modern packaging of "establish a shared secret with someone using only their public key." The sender feeds in the recipient's public key and gets back two things: a random secret key K, and a *ciphertext* — a sealed envelope that the recipient (and only the recipient) can open to recover the same K. Think of it as a one-shot, one-way handshake: no interaction with the recipient needed. Classical systems do this with Diffie–Hellman on the curve; Botho does it with a post-quantum KEM (Section 6).

That's the toolbox. Everything else is assembled from these four parts.

---

3. Hiding the recipient: stealth addresses

The problem with addresses

Suppose your address is public — printed on your website for donations. On a transparent chain, every donation is visibly *to that address*, and your entire donation history is public. Reusing an address links everything sent to it.

The fix sounds paradoxical: every payment goes to a fresh address that the recipient never had to hand out. The sender *derives* a brand-new, one-time address from the recipient's published address, in such a way that:

- outside observers cannot connect the one-time address to the published one,
- the recipient can *detect* "that output is mine" while scanning the chain, and
- only the recipient can *spend* it.

This is a stealth address. A helpful analogy: your published address is not a mailbox, it is a *recipe for making mailboxes*. Each sender uses the recipe to build a fresh, unmarked mailbox that only you have the key to, placed in a public square full of millions of identical unmarked mailboxes.

The two-key wallet

To make "detect" and "spend" separable, a Botho wallet has two key pairs rather than one:

- a view key — used to *recognize* incoming outputs while scanning the chain, and
- a spend key — required to actually *spend* them.

The published address is essentially the pair of public keys. The separation is deliberately useful: you can hand your view key to an accountant or an auditor, who can then see your incoming payments but cannot touch the money. Both keys, and everything else in the wallet, are derived from a single 24-word mnemonic phrase, so one backup recovers everything. Botho also supports *subaddresses* — unlimited unlinkable published addresses from the same wallet — for keeping, say, your store receipts and your donations unconnectable even to your own payers.

How the classical construction works (one paragraph, no equations)

In the classical CryptoNote design, the sender performs a Diffie–Hellman handshake against the recipient's view key to obtain a shared secret, and combines that secret with the recipient's public spend key to mint the one-time output key. The recipient, scanning the chain, redoes the same handshake from their side for each new output and checks "does this output key match what *I* would derive?" A match means "mine," and the matching math also hands the recipient the one-time *private* key needed to spend it later. Everyone else sees only an unlinkable random-looking key.

Botho keeps this exact shape but replaces the Diffie–Hellman handshake with a post-quantum one — that upgrade is Section 6, after you have seen the rest of the classical machinery.

> For the formal construction — key hierarchy, subaddress derivation, and the security theorems — see §4 of the whitepaper (Cryptographic Protocol), subsections "Key Hierarchy" and "Post-Quantum Stealth Addresses."

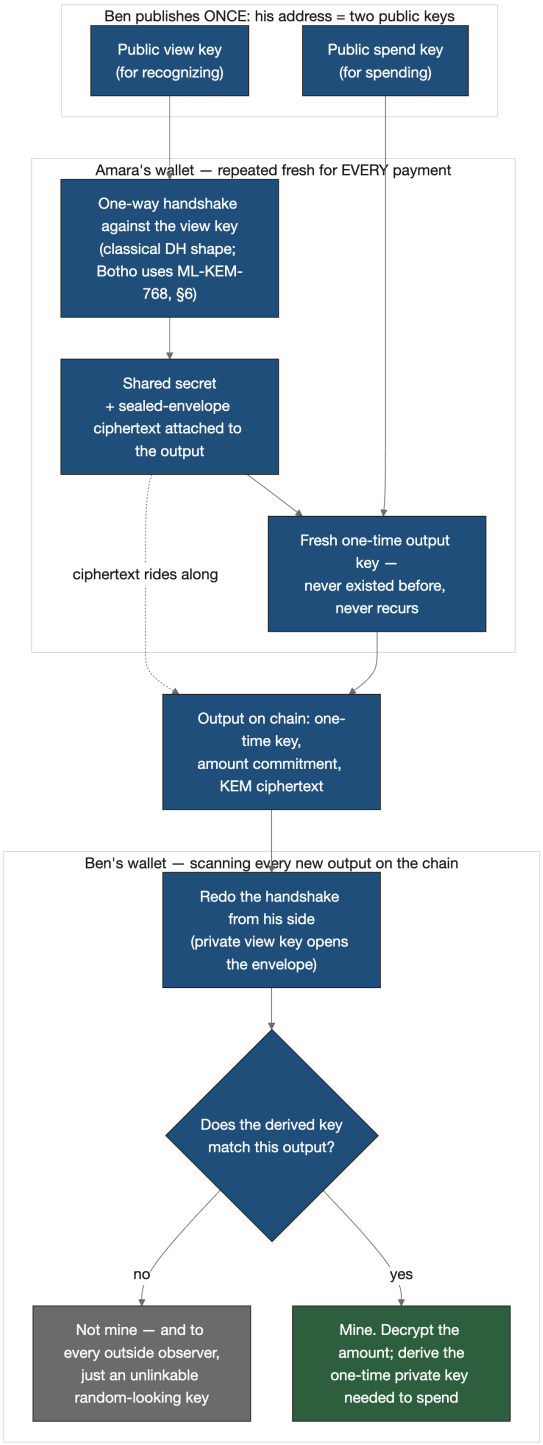

Figure 1 — The stealth-address flow. The sender derives a fresh one-time output key from the recipient's two published keys via a one-way handshake (classical Diffie–Hellman in shape; ML-KEM in Botho, Section 6); the recipient redoes the handshake with the view key to recognize "mine"; everyone else sees only an unlinkable random-looking key.

4. Hiding the sender: ring signatures

The idea: sign as a crowd

A normal signature says "I, key X, approved this." That is exactly what we must avoid — it would announce which coin is being spent, and coins trace back to their history.

A ring signature instead proves: "one of these 20 keys approved this, and I won't tell you which." When you spend an output in Botho, your wallet picks 19 other outputs from the chain's history as decoys, forms a *ring* of 20 candidate outputs (yours hidden among them, at a secret position), and produces a signature that verifies against the whole ring. A verifier learns that the spender controls *one* ring member — with no information about which one. The decoys' owners are never involved and never know their outputs were used.

The analogy that holds up: a petition signed in a sealed voting booth by exactly one member of a named committee of 20. Everyone can verify a committee member signed; nobody can tell which.

Botho uses CLSAG (Concise Linkable Spontaneous Anonymous Group signatures), the same scheme adopted by Monero — compact (roughly 700 bytes per input) and battle-tested. "Spontaneous" means the signer needs no cooperation from the decoys; "linkable" is the next subsection.

The double-spend problem, and the key image

Hiding which coin was spent creates an obvious hole: what stops you from spending the same coin twice, hidden in two different rings?

The answer is the key image: a value deterministically derived from the one-time private key of the *real* output being spent, published alongside every ring signature. Two properties make it work:

1. Same coin ⇒ same key image. Spending the same output twice necessarily produces the same key image, and every node keeps a database of all key images ever seen. A repeat is rejected on sight.
2. The key image doesn't reveal the coin. It is derived through one-way operations, so observers cannot connect it back to any ring member.

So a key image is like a serial number that is *unique to the coin* but *unlinkable to the coin* — a fingerprint that betrays repetition without betraying identity. The ring signature internally proves that the key image was correctly formed from the real ring member, so you cannot lie about it.

Choosing decoys well

Decoy selection matters more than it first appears: if decoys were chosen naively (say, uniformly across all history), statistical patterns would give away the real member — real spends skew heavily toward recent outputs. Botho's wallet therefore samples decoys to match the empirically observed spend-age distribution, and also requires decoys whose *cluster tags* (Section 9 — a Botho-specific notion of coin ancestry) resemble the real input's, so the tags don't fingerprint the real member either.

An honest caveat

Ring signatures give *probabilistic* anonymity: you hide in a crowd of 20, not in the whole universe of outputs, as Zcash's shielded pool allows. Botho accepts this trade deliberately — ring signatures are small, fast, need no trusted setup, and their failure modes are well understood after a decade of Monero

production use. The whitepaper's §2 (Related Work) lays out this comparison honestly, including what Zcash's approach does better.

> For the CLSAG signature equations, the unforgeability/anonymity/ linkability games and proofs, see §4 of the whitepaper, subsection "Ring Signatures (CLSAG)"; ring-size rationale is in the Parameter Justification appendix.

---

5. Hiding the amount: commitments that still add up

The trick: lock the number in a box the math can still see

Hiding amounts sounds impossible: if amounts are hidden, how does anyone verify that a transaction doesn't create money out of thin air?

The tool is a Pedersen commitment. Committing to an amount is like putting the number in a locked, opaque box and bolting the box to the transaction: the box hides the number (in fact *perfectly* — even unlimited computing power cannot extract it, because every possible amount is consistent with what's visible), yet the committer cannot later claim a different number was inside (changing it would require breaking the one-way curve math). Each commitment also folds in a random blinding factor — noise that makes two commitments to the same amount look completely different.

The magic property is that these boxes are homomorphic: you can add two boxes together and get a box containing the sum, *without opening either*. That single property rescues verification. A transaction proves it conserves value by showing:

> (sum of input boxes) = (sum of output boxes) + (fee, in the clear).

Validators check this equation directly on the boxes. If it balances, no money was created or destroyed — and nobody learned any individual amount. Fees stay public so the equation has a public anchor (and, as you'll see in Section 9, because fees are doing economic work in Botho).

The blinding factor for each output is derived from the same shared secret as the stealth address, and the amount itself rides along in a small encrypted field, so the recipient can open their own box — and only theirs.

The hole, and the patch: range proofs

The balance equation has a subtle exploit. Amounts live in modular arithmetic, where numbers wrap around — so a "negative" amount is indistinguishable from an astronomically large one. A transaction with outputs of $+1000$ and -990 would balance against an input of 10… and the -990 output, viewed another way, is a gigantic positive number. Free money.

The patch is a range proof: each output carries a zero-knowledge proof that the committed amount lies in a sane range (0 to 2^{64}), without revealing it. Botho uses Bulletproofs, whose size grows only logarithmically — proofs for many outputs aggregate into barely more space than a proof for one. This is standard, well-studied machinery; if you want the internals, the Bulletproofs paper and *Zero to Monero* cover it far better than we could here.

With this third tool the classical privacy triad is complete: stealth addresses hide *to whom*, ring signatures hide *from whom*, commitments plus range proofs hide *how much*.

> For the formal treatment — commitment properties, the value-conservation equation, and range-proof requirements — see §3 (Preliminaries, "Pedersen Commitments," "Bulletproofs") and §4 (Confidential Transactions) of the whitepaper.

---

6. The quantum problem, and Botho's hybrid answer

This is the first of the three novel-to-Botho sections. There is no Monero tutorial to defer to here.

Why "we'll upgrade later" doesn't work for a blockchain

Recall the toolbox: all classical public-key cryptography rests on one-way curve multiplication, and a large quantum computer running Shor's algorithm undoes it. For most of the internet this is survivable — TLS sessions are ephemeral; when the threat nears, you rotate algorithms and old traffic is gone.

A blockchain has no such luxury. The chain is a permanent public archive, being copied by anyone who wants, *today*. Whatever secrets its recorded data protects are only as safe as the cryptography will be at any point in the *future*. This is the "harvest now, decrypt later" threat from Section 1, and it slices Botho's data into two classes:

- Data whose secrecy must last forever. The stealth-address handshake is the prime example: if a future quantum computer can redo the Diffie–Hellman handshakes recorded on a classical chain, it can retroactively answer "which outputs belong to which published address?" for *all of history*. Recipient identity — who owns what — stays valuable indefinitely.
- Data whose secrecy decays. Sender anonymity is the example: learning in 2045 who sent a particular payment in 2025 has sharply diminished value. The goods shipped, the context evaporated.

Botho's design rule follows directly: permanent secrets get post-quantum protection; ephemeral secrets get efficient classical protection. The whitepaper calls this the hybrid architecture, and it is a considered trade-off, not a compromise of convenience.

The post-quantum stealth address: ML-KEM-768 replaces the handshake

NIST's post-quantum standardization produced ML-KEM (formerly known as Kyber), a lattice-based key-encapsulation mechanism — exactly the "sealed envelope handshake" black box from the toolbox, built on math that resists both classical and quantum attack. Botho uses the ML-KEM-768 parameter set (NIST's middle security category).

Now recall the *shape* of the classical stealth address from Section 3: sender does a handshake against the recipient's view key → shared secret → one-time output key; recipient redoes the handshake to recognize and spend. Botho keeps that shape and swaps only the handshake step:

- Sender: instead of curve Diffie–Hellman, run ML-KEM encapsulation against a KEM public key derived from the recipient's view key. This yields the shared secret plus a ciphertext (the sealed envelope). The shared secret drives the one-time output key exactly as before; the ciphertext (1,088 bytes) is attached to the output.
- Recipient: for each new output on the chain, decapsulate the attached ciphertext with the KEM secret key (derived from the view key), recompute the expected one-time key, and check for a match. A match means "mine," and yields the one-time private key for later spending.

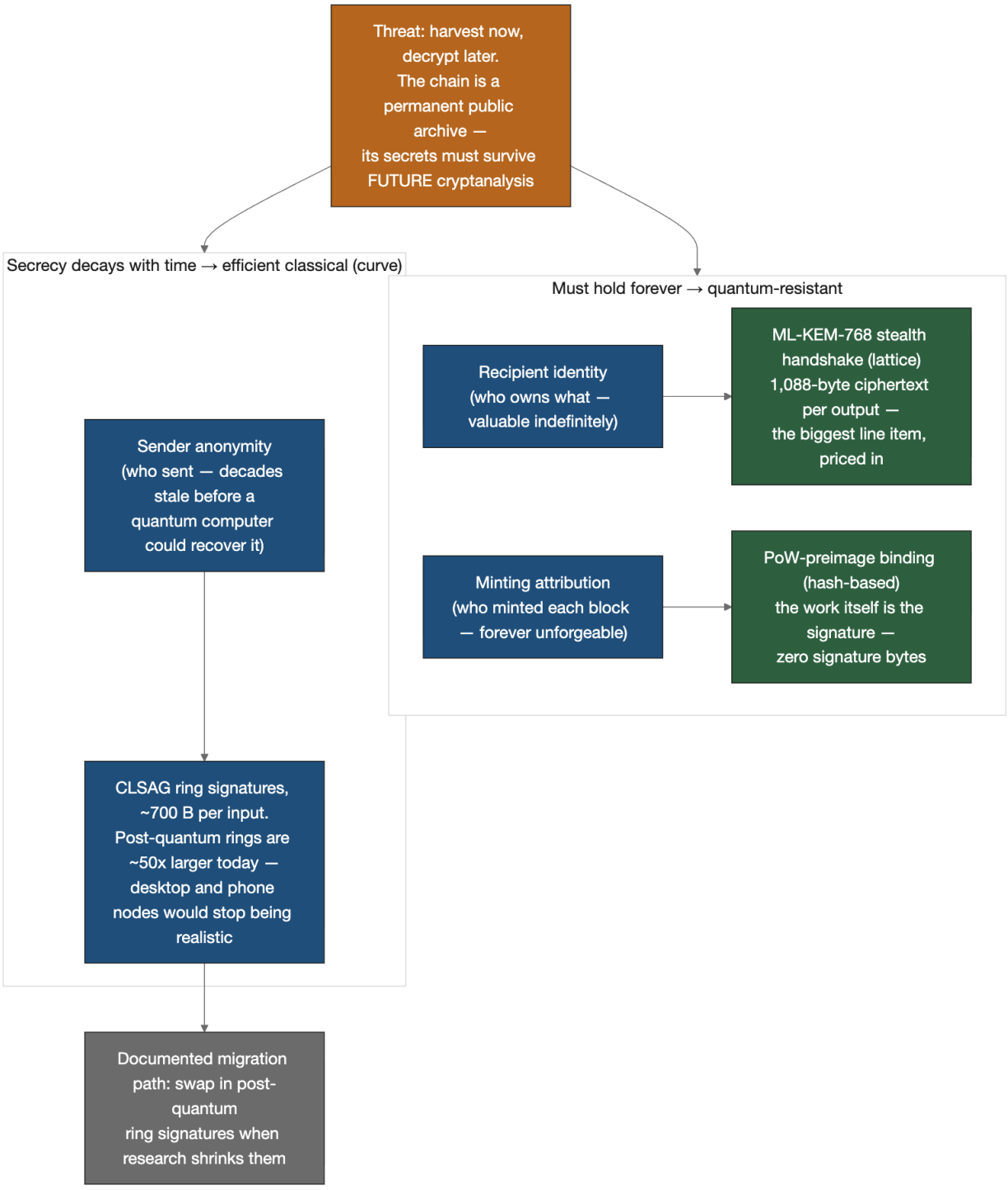

Figure 2 — The hybrid architecture under "harvest now, decrypt later." Guarantees that must hold forever get quantum-resistant protection: recipient identity rides the lattice-based ML-KEM-768 handshake, and minting attribution is bound by the hash-based proof-of-work preimage itself — no signature at all; sender anonymity, whose value decays with time, stays on compact classical CLSAG rings, with a documented migration path.

The consequence: even a quantum adversary armed with the entire recorded blockchain cannot link outputs to recipients, because recipient privacy no longer rests on curve math at all. The whitepaper proves this by reduction to ML-KEM's standard security property.

The visible cost is size — 1,088 bytes of ciphertext per output, the biggest single line item in a Botho transaction. That is the price of future-proofing the data that lives forever.

Minting attribution: the work itself is the signature

One more permanent-data case: block rewards. When a miner mints new coins (Section 8), the minter's identity is deliberately public, and that attribution must remain unforgeable for the life of the chain — nobody should ever be able to retroactively forge "I minted that block." Your first instinct is probably "so sign it, with a post-quantum signature" — and that instinct is expensive: a lattice signature plus its public key would add roughly 5.3 KB to every block, forever.

Botho's design needs no signature at all, because the proof of work already does the job. The block of data a miner hashes over and over in the mining race (Section 8) *includes the miner's own public address keys*. So the winning hash is inseparable from the winner's identity: to reattribute an already-mined block, you would have to swap the keys inside the hashed data — which changes the hash, which destroys the very proof of work that made the block valid — and then redo the mining race against a block the network has already finalized. The work itself is the signature. And unlike curve math, this binding rests entirely on hashes, which a quantum computer cannot invert with Shor's algorithm — it gets only a modest generic search speedup (Grover's algorithm), nothing like an outright break. Minting attribution is therefore quantum-safe *for free*: zero signature bytes per block, roughly 5.3 KB saved every block versus the signature route. (One lattice signature scheme does keep a named role in the design: ML-DSA-65, NIST's post-quantum signature standard, is the designated signature family *if* a post-quantum authorization path is ever introduced in a future protocol version.)

Why ring signatures stay classical

The natural question: why not make *everything* post-quantum? Because post-quantum ring signatures are, today, brutally large — on the order of 50× a CLSAG. A ~700-byte CLSAG becomes ~35 KB per input; a routine several-input payment would exceed 100 KB, and running an ordinary desktop or phone node would stop being realistic. And the secret a ring signature protects — *who sent this* — is precisely the ephemeral kind: by the time a quantum computer exists to break today's rings retroactively, the information it recovers will be decades stale.

So the ledger's permanent secrets (who owns what; who minted what) are quantum-safe now, while sender anonymity rides efficient classical machinery — with a documented migration path to swap in post-quantum ring signatures when research shrinks them to practical size.

One intuition worth carrying away precisely: the hybrid design does *not* mean "Botho is partly quantum-safe and hopes for the best." It means the quantum exposure is confined to the one secret whose value provably decays with time, and that confinement was chosen, argued, and priced.

> For the formal treatment — protocol listings, the recipient-unlinkability theorem and its QROM reduction, the data-lifetime table, and the full-post-quantum cost analysis — see §4 of the whitepaper ("Post-Quantum Stealth Addresses," "Minting Attribution," "Hybrid Architecture Rationale") and the post-quantum survey in §2.

---

7. Agreement without a leader: the Stellar Consensus Protocol

Novel-to-Botho piece two spans this section and the next: Botho pairs SCP consensus with proof-of-work mining, a combination essentially no other deployed system uses. This section teaches SCP; the next teaches the pairing.

What consensus is actually for

Thousands of independent nodes each hold a copy of the ledger, and new transactions arrive at all of them in different orders. Consensus is the process by which they all commit to *the same next block* — even though some nodes are offline, some are slow, and some may be actively lying.

Bitcoin's answer (Nakamoto consensus) is "the chain with the most accumulated work wins." It is beautifully permissionless, but the agreement is only ever *probabilistic*: a block can always, in principle, be displaced by a longer competing chain, which is why exchanges wait for confirmations and finality takes tens of minutes.

The classical alternative — BFT (Byzantine-fault-tolerant) voting protocols, the family of agreement algorithms built to survive participants that fail or lie — gives *deterministic* finality (once committed, committed forever) but traditionally requires a fixed, globally known list of validators, which is the opposite of permissionless.

Quorum slices: trust chosen locally, agreement achieved globally

SCP (the Stellar Consensus Protocol) is a BFT-family protocol that removes the fixed membership list. Its core idea is the quorum slice: each node *individually* declares which sets of other nodes it trusts — "I will accept a statement if all of these accept it." Nobody hands out a global roster; each operator makes a local, subjective choice, the way you choose which DNS resolvers or package mirrors to trust.

Out of everyone's overlapping local choices, global structures emerge: a quorum is a set of nodes that is self-sufficient — every member finds a full slice of its own inside the set. When a whole quorum agrees on a statement, every member is convinced by nodes it personally trusts.

The safety condition is quorum intersection: any two quorums must share at least one honest node. Picture two committees that both claim authority to ratify a decision: if every pair of possible committees is guaranteed a common honest member, they can never ratify contradictory decisions — that shared member would have had to vote both ways, and honest nodes don't. From this single overlap property, SCP derives its headline guarantee: no two honest nodes ever finalize different blocks at the same height. In Botho, trust choices are the operators' (with a sensible tiered default — a few high-uptime infrastructure nodes plus community validators — shipped out of the box).

Rounds: nominate, ballot, externalize

Mechanically, each block height runs a short protocol:

1. Nomination — nodes propose and converge on a candidate block for the slot.
2. Ballot — nodes run prepare/commit voting rounds on the candidate (with escalating ballot numbers if a round stalls).
3. Externalize — once a quorum commits, the block is *final*. Not probably-final: final. There are no reorgs of externalized blocks, no confirmation counting.

The whole voting sequence takes a few seconds after a block is proposed.

The deliberate failure mode: halt, don't fork

What if the network partitions, or too many trusted nodes go down and no quorum can form? SCP — and Botho with it — makes a conscious choice: stop producing blocks rather than risk two histories. Safety over liveness. For money, this is the right side of the trade: a currency that pauses is an inconvenience; a currency that forks into two versions of who-owns-what is a catastrophe. When the partition heals, progress resumes from where the ledger halted — nothing needs to be unwound.

> For the formal treatment — quorum slice/quorum/intersection definitions, the four consensus phases, the fork-freedom theorem and its proof, and liveness assumptions — see §6 of the whitepaper (Consensus Mechanism). The original SCP paper (Mazières) is the external deep dive.

---

8. Mining without voting: proof-of-work, decoupled from consensus

If SCP finalizes blocks, why mine at all?

MobileCoin — the closest prior design, pairing CryptoNote privacy with SCP — answers "don't": it has no mining and a fixed, fully pre-created supply, with validators finalizing fee-paying transfers. Botho answers differently, and the reason is worth understanding because it explains an unusual architecture.

SCP is a mechanism for *agreement*, not for *issuance*. It says nothing about where new coins come from or who gets them. If you want new currency to enter circulation *permissionlessly* and *fairly* — earned by anyone willing to contribute a real, measurable resource, rather than allocated by founders or by whoever already holds coins — you still need something like proof-of-work. So Botho splits the job in two:

- Proof-of-work decides who gets to propose the next block — and who earns the new coins in it. Miners race to find a block-header nonce — a throwaway counter they vary until the whole header's hash falls below a difficulty target. Recall from Section 2 that hash outputs are unpredictable, so the only way to win is brute trial: finding such a hash is provable evidence of work done. Winning this race is a *lottery ticket weighted by computation*: it buys you the right to put a candidate block on the table and to claim the block reward if it's chosen.
- SCP decides which proposed block becomes final. The quorum machinery from Section 7 picks among valid proposals and externalizes exactly one.

A proposed analogy: mining is *applying to speak*; consensus is *the room agreeing to enter your words into the minutes*. Hashpower gets you to the podium more often. It does not give you any say over what the room accepts.

The load-bearing design decision: mining weight ≠ consensus weight

In Bitcoin, hashpower is voting power: 51% of the hashrate can rewrite recent history. In Botho, hashpower buys *zero* votes. Consensus weight lives entirely in the quorum-slice trust graph — in which nodes operators have chosen to trust — and a warehouse of miners has exactly as much say over finality as it has earned places in other operators' slices: none, by default.

This decoupling buys three things:

1. Hashpower majorities can't rewrite history. A 51% attacker in Botho can win block proposals more often, but externalized blocks are final; rewriting them would require corrupting the quorum structure itself — a completely different (and social, not purchasable) attack surface. Buying hardware doesn't buy the ledger.
2. Deterministic finality at mining speed. Blocks are final seconds after proposal — block time plus a few seconds of SCP voting — instead of after six confirmations.
3. Energy goes to distribution, not to security-by-burn. PoW's job narrows to metering out new coins fairly and keeping proposal permissionless. It does not have to carry the entire security budget on its back, which is what makes the modest emission schedule of Section 9 viable.

The converse discipline also matters: coin *ownership* buys no consensus power either (this is not proof-of-stake), and validators earn no fees — so there is no path by which either wealth or hashpower quietly accumulates governance.

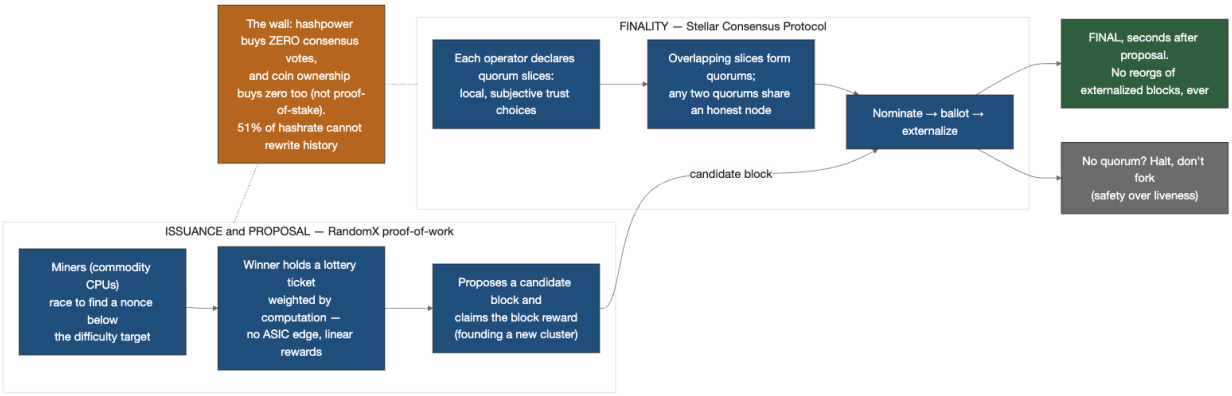

Figure 3 — The wall between issuance and finality. RandomX proof-of-work meters out new coins and the right to propose blocks; operator-chosen quorum slices give SCP its deterministic finality (halt, don't fork). Hashpower buys zero consensus votes — a candidate block crosses the wall, voting power never does.

RandomX: keeping the mining lottery egalitarian

Botho's proof-of-work uses RandomX, the CPU-oriented algorithm developed for Monero. RandomX is deliberately hostile to ASICs (application-specific integrated circuits — chips custom-built for a single algorithm): it executes randomly generated programs and leans on general-purpose CPU features (caches, branch prediction, wide memory), so a commodity CPU is close to the efficient frontier. The intent is that the issuance lottery stays open to ordinary hardware — a laptop or a $50/month cloud node buys tickets at odds proportional to the computation it contributes, with no structural edge for specialized hardware — rather than collapsing into a specialized- hardware oligopoly. Mining rewards scale linearly with hashpower (no economies of scale in the protocol itself), and because mining carries no consensus weight, even a mining-pool concentration is an economics problem, not a ledger-integrity problem.

One more Botho-specific wrinkle worth knowing as a node operator: block timing is adaptive. The target interval stretches and shrinks with transaction load — from 3 seconds under heavy use out to 40 seconds when the network is idle, with 5 seconds as the reference pace under sustained load. Since each block carries a reward, an idle network automatically mints fewer coins per day than a busy one. This "emission breathes with usage" behavior is really a monetary mechanism, so it is covered with the

economics in the next section.

> For the formal treatment — the design-rationale comparison of pure PoW vs. pure BFT, difficulty adjustment, fork-freedom and its corollaries, nothing-at-stake and long-range-attack analyses, and mining-pool considerations — see §6 of the whitepaper; RandomX and decentralization incentives appear in §7 ("Minter Incentives") and the block-time discussion in §7 ("Dynamic Block Timing").

---

9. Money that dislikes hoarding

The third novel-to-Botho piece, and the whitepaper's most original contribution. Take this section slowly; each mechanism exists to patch a specific hole in the previous one.

9.1 The supply: a five-year distribution, then a 2% heartbeat

New coins enter only as block rewards. The schedule has two phases:

- Phase 1 (about five years): rewards start at 50 BTH per block and halve each year — 50, 25, 12.5, 6.25, 3.125. At the 5-second reference pace the chain produces about 6.3 million blocks a year, so year one distributes roughly 315 million BTH, and each following year half as much — summing to roughly 611 million BTH across the first five years. That is the large majority of supply for the chain's first decades (the tail emission below keeps adding forever; twenty years in, Phase-1 coins still make up about 69% of it). Compare Bitcoin, which stretches halvings over decades and thus concentrated its cheap coins in a tiny early cohort; Botho's compressed schedule is a deliberate fairness choice — the distribution window is short, then over.
- Phase 2 (forever): a perpetual tail emission targeting about 2% net annual supply growth. Not a fixed trickle of coins — the tail scales with supply so the *rate* stays pinned at ~2% (slightly more is emitted gross, anticipating the fee burning described below).

Why perpetual inflation, when "sound money" orthodoxy says cap it? Two reasons you have already met. First, security: miners must be paid forever, and 2% emission funds them without praying that fee markets suffice (this is Bitcoin's open long-term question). Second — and this is where Botho's philosophy bites — mild inflation *is the point*: roughly 2% annual dilution acts as a gentle wealth tax on idle balances. Money sitting still slowly cedes ground; money circulating in an economy earns its keep. And remember from Section 8: because block timing adapts to load, an idle network emits *less* than the target — inflation throttles itself when nobody is transacting.

The base unit, for the record: 1 BTH = 10^{12} picocredits; protocol arithmetic is integer picocredits throughout.

9.2 The hard problem: how do you tax wealth you can't see?

Here is the puzzle that makes Botho's economics genuinely novel. A progressive system needs to treat large holders differently from small ones. But Botho just spent five sections making sure *nobody can see who holds what*: amounts are hidden, addresses are one-time, ownership is unknowable. Worse, even without privacy, "one holder" is not observable on any blockchain — a whale can split into a million wallets for free. Any mechanism keyed to *identity* or to *account structure* is dead on arrival.

Botho's answer: don't tax the holder — tax the coins' lineage. Coins themselves carry ancestry, and ancestry is the one thing a holder cannot fake, split away, or hide, because it travels with the value itself.

9.3 Cluster tags: provenance that survives splitting

Every minting event (every block reward) founds a new cluster — think of it as a family line of coins, identified by a hash of the minter's key and the block height. The freshly minted coins are tagged 100% with that new cluster.

From then on, every coin (every UTXO — an unspent transaction output, the discrete "coin" objects a wallet holds) carries a tag vector: a weighted breakdown of which clusters its value descends from. When a transaction merges inputs, output tags are the value-weighted blend: spend 70 BTH of pure cluster A with 30 BTH of pure cluster B, and every output is tagged 70% A / 30% B.

Watch what this does to the obvious attacks:

- Splitting is a no-op. Divide one output into a thousand; each child inherits the parent's tag vector *unchanged*. Provenance is a fraction of value, not a property of the container, so re-slicing the value re-slices nothing.
- Shuffling among your own wallets is a no-op. Self-transfers blend your tags with your own tags.
- The only way to genuinely dilute a tag is real commerce — acquiring differently-tagged coins from other people, which means giving up value in exchange.

Each cluster's wealth is tracked as the total value across the whole UTXO set attributed to it — a lineage that has accumulated a lot of value scores high, whether it sits in one output or a million. From a coin's tag vector and those cluster wealths, the protocol computes the coin's cluster factor: a number from 1× to 6× that answers, roughly, "how wealthy is the lineage this coin descends from?" Coins from small, diffuse lineages sit at 1×; coins descending from lineages holding on the order of 100,000 BTH sit mid-scale (3.5×); coins from million-BTH lineages climb past 5× toward the 6× ceiling. (The exact curve is a log-domain sigmoid — see the whitepaper; the intuition is that the factor climbs across *orders of magnitude* of lineage wealth, so it distinguishes "person" from "whale," not "person" from "slightly richer person.")

One valve prevents tags from being a life sentence: tags decay by 5% per qualifying transfer — but a coin must age about an hour (720 blocks at the reference pace) before a transfer qualifies. So genuine circulation gradually washes coins clean over weeks of real economic movement, while rapid self-churning achieves nothing: the clock, not the transaction count, is the binding constraint.

9.4 Progressive fees: the intake side

Now the tax code writes itself. Every transaction's minimum fee is scaled by the cluster factor of the coins being spent: factor-1 coins pay the base rate; factor-6 coins pay six times as much. Small users pay essentially nothing — the floor fee for a typical transfer is measured in *nano*-BTH, deliberately negligible — while movements of concentrated-lineage wealth pay up to 6× that (still tiny in absolute terms; the point is pressure, not punishment).

Because the fee keys on provenance rather than identity, it is Sybil-proof by construction: the whitepaper proves that no strategy of splitting, shuffling, or address-spawning lowers your total fees — every child coin carries the ancestry, and blending only ever *averages* factors, never launders them. There's a subtle interaction with privacy worth admiring: ring signatures hide *which* ring member is really being

spent, so which factor applies? Botho charges the maximum factor among the ring members — and recall from Section 4 that decoys must have tags similar to the real input's, so this maximum is honest and whales can't hide behind low-factor decoys.

Two more pieces ride on the same machinery:

- Demurrage — the holding-cost term. Transaction fees only bite when coins *move*; wealth that sits still would escape entirely. So the fee floor includes a stock-level term: when high-factor coins finally move, they owe a charge proportional to the value moved, the time held, and the cluster factor — in effect a parking fee accrued by idle concentrated wealth. To give that a size: at the whitepaper's representative operating point, a factor-6 lineage that sat idle for a year owes on the order of 2% of the value it finally moves — a mid-scale factor-3.5 lineage roughly half that — modest in any one year, but it keeps accruing for as long as the coins sit idle. Factor-1 coins are exempt: ordinary users never pay it. (This is a pointed refinement of the century-old "demurrage currency" idea — Gesell's stamp scrip taxed *everyone's* holdings; Botho's version taxes only concentrated lineages.)
- Congestion pricing. The per-byte base fee floats upward under load (bounded at 100× the floor), a standard anti-spam valve, orthogonal to the progressive machinery.

9.5 Where the fees go: burn a fifth, lottery the rest

In Bitcoin, fees go to miners — which invites fee-market manipulation and the transaction-reordering games known as miner-extractable value (MEV). In Botho, miners never receive fees (they are paid purely by block rewards, which is why they have no motive to reorder or censor anything). Instead, every block's collected fees are split:

- 20% is burned — destroyed, shrinking supply, a small deflationary rebate to every holder equally. Under heavy use, burning can even outpace tail emission.
- 80% goes into the lottery pool, along with something more important: a scheduled slice of the block reward itself, ramping up over the first halvings to 50% of each reward. This emission routing is crucial — fees are a tax on *transacting*, so fees alone could never reach wealth that simply sits; routing new emission into the pool is what makes the redistribution reach idle wealth too.

Each block, the pool pays out (capped at one block reward per block; surplus carries over) to four randomly selected UTXOs — ordinary coins sitting in ordinary wallets, no registration, no staking, no action required. Your wallet can simply be a little heavier one morning.

The selection is where all the machinery converges. A coin's chance of winning is proportional to its value times an inverse-factor tilt: a factor-1 coin gets six times the winning weight per BTH of a factor-6 coin. Both properties are load-bearing:

- Value-weighted ⇒ splitting your money into a million UTXOs does not change your total odds by one iota. (Eligibility has a dust floor — a millionth of a BTH — and a maturity age, ~an hour, to keep the candidate set sane.)
- Inverse-factor tilt ⇒ per unit of value, the flow runs *toward* coins from small lineages — toward commerce, wages, and ordinary balances, and away from whale lineages.

Net effect: value flows continuously from the fees and diluted holdings of concentrated wealth toward a random scatter of small holders — a redistribution engine with no tax authority, no identity, and no way to game it by restructuring.

And the randomness itself can't be gamed: each block's lottery seed is derived from the hash of the *previous, already-finalized* block, so a block's proposer cannot bias a draw by fiddling with the transactions in the block it is building. The only lever left — re-grinding the previous block's proof-of-work to reshape its hash — costs a full PoW solution (worth one block reward) to redirect at most a fraction of the capped payout. The attack costs more than it can ever win, by construction.

9.6 Why not the simpler designs? (The graveyard that shaped this one)

The design above looks baroque until you see what fails without each piece. The whitepaper reports adversarial simulations — every candidate mechanism was tested against a strategic whale playing the optimal splitting-and-churning strategy — and three results explain the shape of the final system:

1. "Pay each UTXO equally" inverts under attack. A per-UTXO lottery is the *best* redistributor against honest participants and the *worst* against a strategist: the whale splits into thousands of UTXOs, captures the payout stream, and inequality *rises*. Moral: never pay out on anything a holder can manufacture for free.
2. Untargeted dilution redistributes nothing. Emission paid out in proportion to existing wealth is exactly neutral — the payout must be *tilted* to change anything.
3. Cluster-anchored mechanisms don't degrade under attack — they profit. Against the full system (tilted lottery + emission routing + demurrage), the strategic whale's splitting and churning just generates fees that feed the pool; in simulation the attack *costs the attacker* roughly a fifth of its position over five years while redistribution holds essentially unchanged — it does not degrade. Attacking the mechanism funds it.

That is the deepest design idea in Botho's economics: in an anonymous system, coin ancestry is the only wealth signal that restructuring cannot forge — so both the intake (fees, demurrage) and the outflow (lottery tilt) anchor to it.

> For the formal treatment: emission schedule and dynamic timing, §7 (Monetary Policy); cluster tags, blending, decay, the factor curve, and the Sybil-resistance theorem, §5 ("Cluster Tags and Progressive Fees"); lottery mechanism and seed, §7 ("Fee Economics"); the adversarial simulations and Gini results, §10 ("Quantitative Economic Modeling"); grinding cost-benefit, §9 ("Lottery Grinding Attack"); parameter rationales, the Parameter Justification appendix.

---

10. Putting it together: one payment, end to end

Amara owes Ben 40 BTH. Here is everything that happens, using only machinery you have now seen. Ben has sent Amara his published address — the two public keys (view + spend) from Section 3 — over any channel; handing out an address reveals nothing to anyone else, ever.

1. Choosing the coins. Amara's wallet holds her UTXOs — say a 30 BTH and a 25 BTH output from earlier transactions. It selects both (55 BTH total) to cover 40 BTH plus fee, planning change back to herself.

2. Pricing the transaction. Both inputs carry tag vectors tracing to modest lineages: cluster factor 1×. The wallet computes the fee: base rate (1 pico-BTH per byte, at the congestion floor) × ~5,000 bytes × factor 1 × a small multi-output penalty (4, for the two outputs) = 20,000 pico-BTH — 20 nano-BTH — with no demurrage term (factor-1 coins are exempt). Had Amara been spending coins descended from

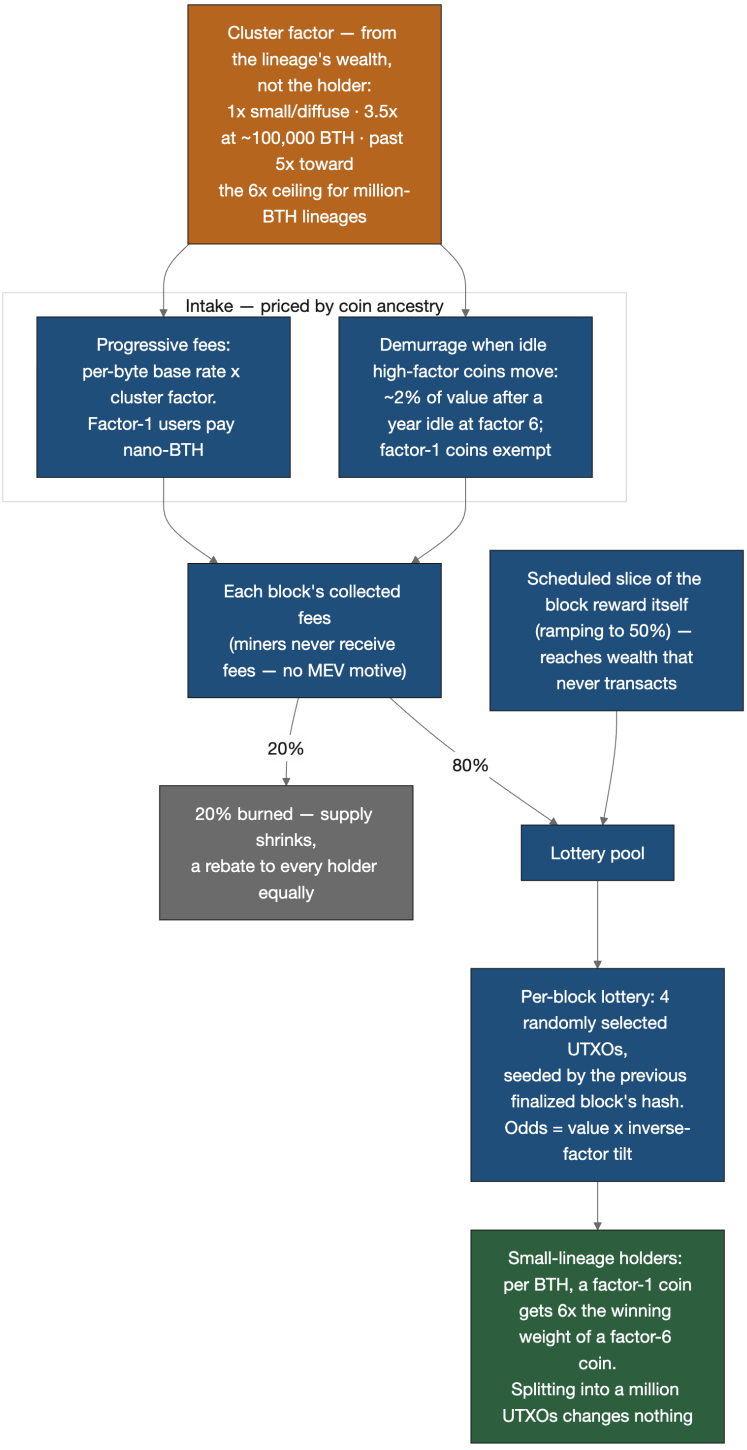

Figure 4 — The anti-hoarding money flow. The cluster factor — computed from coin lineage, not holder identity — prices the intake (progressive fees plus demurrage on idle concentrated wealth); of each block's fees, 20% is burned and 80% joins a scheduled slice of the block reward in the lottery pool, which pays four random UTXOs per block at odds of value times an inverse-factor tilt toward small lineages.

Figure 5 — The whole payment at a glance: Amara's wallet builds the transaction (steps 1–6), the network propagates and validates it (step 7), a miner proposes and SCP finalizes (steps 8–9), and the aftermath plays out — lottery drawn, Ben's wallet finds its money (steps 10–11).

a 100,000-BTH lineage, the same transfer would cost ~3.5× more and carry a holding-cost term; the mechanics would otherwise be identical.

3. Building Ben's stealth output (Section 3 + 6). The wallet runs ML-KEM encapsulation against Ben's view key, obtaining a shared secret and a 1,088-byte ciphertext. From the secret plus Ben's spend key it derives a fresh one-time output key — an address that has never existed and will never recur. The 40 BTH amount goes into a Pedersen commitment (blinding factor derived from the same shared secret), with the exact amount tucked into a small encrypted field only Ben can read. A second output — the ~15 BTH change — is built the same way, addressed to Amara herself; on-chain, payment and change are indistinguishable.

4. Proving the amounts are sane (Section 5). The wallet attaches an aggregated Bulletproof covering both output commitments: each committed amount is in range, no negative-value trickery. Validators will check that input commitments = output commitments + the public fee — the books balance without any amount being visible.

5. Hiding the senders (Section 4). For each of the two real inputs, the wallet samples 19 decoy outputs from chain history — age-realistic, with tag vectors similar to the real input's — and signs with CLSAG over each ring of 20. Each signature includes the input's key image. The fee was computed against the highest cluster factor in each ring, so the decoys' presence gave nothing away and saved nothing.

6. Tagging the outputs (Section 9). Both outputs inherit the value-weighted blend of the inputs' tag vectors. Ben's coins will carry Amara's coins' ancestry forward — at factor 1, a fact neither will ever notice in fees.

7. Into the network. The wallet submits the ~5 KB transaction. It propagates via Dandelion++: first a "stem" phase, passed quietly along a random path of nodes, then a "fluff" phase of ordinary flooding — so observers watching traffic cannot triangulate which IP address originated it (see §8 of the whitepaper). Nodes validate: key images fresh (no double spend), rings valid, signatures verify, commitments balance, proofs check, fee sufficient.

8. Mining (Section 8). The network has been moderately busy, so the adaptive target block time is sitting near the 5-second reference. A miner — maybe a hobbyist CPU node — wins the RandomX race, assembles a block containing Amara's transaction plus a minting transaction paying the block reward (no signature needed: the miner's identity is bound by the PoW preimage itself, per Section 6 — founding a brand-new cluster for the fresh coins, with a scheduled slice of the reward routed to the lottery pool), and broadcasts it.

9. Finality (Section 7). SCP takes over: nodes nominate the proposal, run the ballot rounds through their quorum slices, and externalize the block — typically a handful of seconds after proposal. The block is now final. Not "probably final after six confirmations" — final. Ben can release the goods immediately.

10. The lottery (Section 9). The block's fees are split — 20% burned, 80% into the pool with the emission share — and four winners are drawn from all eligible UTXOs on the chain, seeded by the previous block's hash, weighted by value × inverse-factor tilt. Some student's factor-1 wallet in another country just got slightly heavier. Neither Amara nor Ben notices; this happens every block, forever.

11. Ben finds his money. Ben's wallet, scanning new blocks, decapsulates each output's ML-KEM ciphertext with his view key, and on Amara's output the derived key matches: *mine*. It decrypts the amount — 40 BTH — and stores the one-time private key it will need to spend. A future quantum

adversary replaying the whole chain still cannot link that output to Ben's address: the handshake was post-quantum.

What an outside observer saw: a transaction spending two inputs, each hidden among twenty candidate coins, to two never-before-seen output keys, in unknown amounts, from an unknown network origin, finalized in seconds — plus a fee that reveals the transaction's size and the *lineage class* of the coins moved (and, for wealthier lineages, how long they sat idle), but never who and never how much. That's the whole design in one artifact: private to observers, honest to validators, progressive to the economy.

---

11. Where to go next

One status note before you go. This primer, like the whitepaper it accompanies, teaches Botho's *ratified target design* (architecture decision record ADR 0006 in the repository). Two pieces are not yet live on the testnet: amounts are still public on chain, and outputs do not yet carry ML-KEM ciphertexts — so if you open today's block explorer and can read every amount, you are looking at an implementation gap, not a different design. And one thing from older project material really was dropped, not deferred: a separate, opt-in "quantum-private" transaction class from early drafts was retired — quantum-durable recipient privacy is delivered universally by Section 6's design, never sold as a tier.

Into the whitepaper. This primer deliberately taught intuition and deferred rigor. The map, by question:

If you want…	Read (whitepaper)
The design philosophy and threat framing	§1 Introduction
Comparisons: Monero, MobileCoin, Zcash, trusted-execution-environment (TEE, secure-enclave) approaches	§2 Related Work
Formal notation and primitive definitions	§3 Preliminaries + Notation appendix
Stealth addresses, CLSAG, commitments — with proofs	§4 Cryptographic Protocol
Byte-level transaction formats, cluster tags, fee theorems	§5 Transaction Format
SCP phases, fork-freedom proof, mining pools	§6 Consensus Mechanism
Emission math, dynamic timing, lottery mechanics	§7 Monetary Policy
Networking, Dandelion++, light clients	§8 Network Protocol
Threat model, attack scenarios, privacy bounds	§9 Security Analysis
Incentive analysis and the adversarial Gini simulations	§10 Economic Analysis
The reference implementation (Rust) and performance	§11 Implementation
Upgrade and governance process	§12 Governance
Why each constant has the value it has	Parameter Justification appendix

Out to the literature for the standard primitives: *Zero to Monero* (stealth addresses, ring signatures, RingCT, in full depth); the CLSAG and Bulletproofs papers; the CryptoNote whitepaper for historical grounding; Mazières' Stellar Consensus Protocol paper for federated Byzantine agreement; NIST FIPS

203 for ML-KEM (and FIPS 204 for ML-DSA, the designated future signature family); the RandomX specification for the mining algorithm.

What you now know. One pass back over the whole structure: Botho hides recipients with stealth addresses, senders with CLSAG rings, and amounts with commitments — the classical triad (Sections 3–5) — then hardens the *permanent* guarantees against quantum harvest — recipient privacy with ML-KEM-768, minting attribution with the hash-based PoW binding itself — while leaving *ephemeral* sender privacy on efficient classical machinery (Section 6). It finalizes blocks with SCP's trust-graph voting — halt, never fork — while using RandomX proof-of-work purely to distribute issuance and block proposals, keeping hashpower forever divorced from consensus power (Sections 7–8). And on top, it runs an economy that leans against hoarding: a five-year distribution then a 2% tail, fees and demurrage keyed to coin *ancestry* — the one wealth signal restructuring cannot forge — funding a per-block lottery tilted toward small holders, with every design choice hardened against the whale strategies that break the naive versions (Section 9). Ten mechanisms, three problems, one transaction that touches them all.